

HD Submission

Alec Stanley (ID: 33963975)

6th June, 2026

Full source code available at: <https://github.com/alecstanley-git/fit1045-hd-project>

1 Program Description

I have built, from scratch, an N-body gravitational simulator in C++. It makes use of a custom physics engine, a custom rendering library, and other custom tools like a JSON loader and various data types (dynamic arrays, vectors). Notably, no OpenGL or established window management/plotting libraries. On that, I'd like to start off by saying my initial goal with this project was to limit myself exclusively to the standard library of C++. Although a very naive goal, I must say it was the main guiding principle by which I developed my program. Throughout development, I rarely ever thought about using external libraries. I believe this greatly enhanced my understanding of the program I was building. In saying that, there are some libraries that I could not build this without:

1. Apple's proprietary AppKit (including Cocoa and Quartz) for building a window, drawing basic shapes, and text on Mac.
2. The Windows Win32 library, which serves the same purpose.
3. The C++ standard library, including iostream, cmath's sqrt and infinity, strings, and a few others. I did **not** use the standard library's vectors or arrays.

Evidently, I designed the program to work *only* on Windows and Mac, to which I have successfully tested it on both platforms.

1.1 Overall architecture

As mentioned, the program relies on a clean separation between the simulation engine, the rendering, and the I/O.

1.1.1 Simulation engine

The simulation relies on a symplectic, time-reversible numerical integrator known as Leapfrog (a kick-drift-kick integrator) which is known to simulate Newtonian gravitational physics very well. It does however, like any Euler-based integration scheme, struggle to resolve close collisions (due to a division by r^2). To fix this, a softening constant ε was introduced such that the distance was 'softened' by calculating the magnitude distance as $\sqrt{x^2 + y^2 + z^2 + \varepsilon^2}$. This makes a negligible difference at regular distances, but helps massively with close collisions. This is a known technique in these kinds of simulations.

The engine also generates 'galaxy rings' according to the literature of Toomre and Toomre (1972) by initialising **massless** tracer particles in circular orbits around central massive bodies. They are given an initial orbital velocity based on their distance from the central star. This is advantageous since massless tracer particles **do not exert gravity**, but are influenced by it. This turns an $O(N^2)$ problem into an $O(M \cdot N)$ problem.

1.1.2 Rendering

The 3D projection was built from scratch without OpenGL by building a view matrix (look-at) and a perspective projection matrix. The calculation takes any point in the world space, multiplies it by

the view matrix, taking the perspective divide, and mapping the normalised coordinates to the screen space. This is the main process. The renderer also:

1. Uses Liang-Barsky line clipping to clip 3D axis lines to within the visual bounding box. This was cited from the Wikipedia page of the same name (yes, I know, but it works).
2. Handles camera movement controls, allowing the user to scroll in and out to zoom, and drag around to pan the camera.

My custom rendering library is separate from my plotting library, but I will describe it here. The plotting library is able to take arbitrary arrays of points in either 2D or 3D space and produce plots that can be drawn directly to the displayed window. The Figure class and its syntax was built to mimic matplotlib and MATLAB which I have previous experience with.

The window management was built across three separate files. An operating-system-agnostic header file, an implementation .mm file written in combined Obj-C and C++ for Mac, and an implementation C++ file for Windows. The implementation files directly interface with AppKit and Win32 respectively to build the window object, allow the system to track mouse clicks and movements, and draw shapes to the screen.

1.1.3 Other features

The program makes use of a custom JSON parser which was built to allow me to easily demonstrate different orbital configurations. My primary motivation here was to implement a way to make the configurations repeatable and quick to set up. This ended up saving lots of time, even in the development phase.

2 Overview

The program opens to the main menu screen (Figure 1) where the user is prompted to either start or quit. Upon pressing start, the user is taken to the next menu which consists of just a few buttons aligned to the left of the screen. There is lots of empty space which soon gets populated by the simulation once it has been set up.

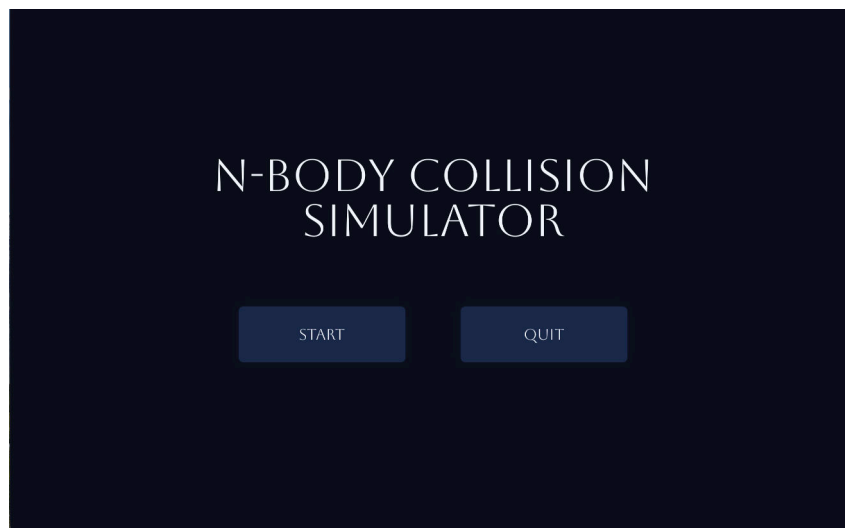


Figure 1: The main menu which displays on first open.

The buttons currently don't yet do anything when clicked. The user first needs to press "Initialise" to begin the setup process. After this, the user is presented with two options, shown in Figure 2. Clicking "Enter new" will prompt the user to enter a new configuration in the console. Clicking "Upload from

file” will ask the user to select from a pre-set list of configurations, which can be adjusted externally. Figure 3 shows the file selection menu. If the user had decided to enter a new configuration, they would be asked:

1. The number of bodies to add
2. For each body:
 - Mass
 - Disk angle
 - Ring number
 - Initial position (x, y, z)
 - Initial velocity (v_x, v_y, v_z)

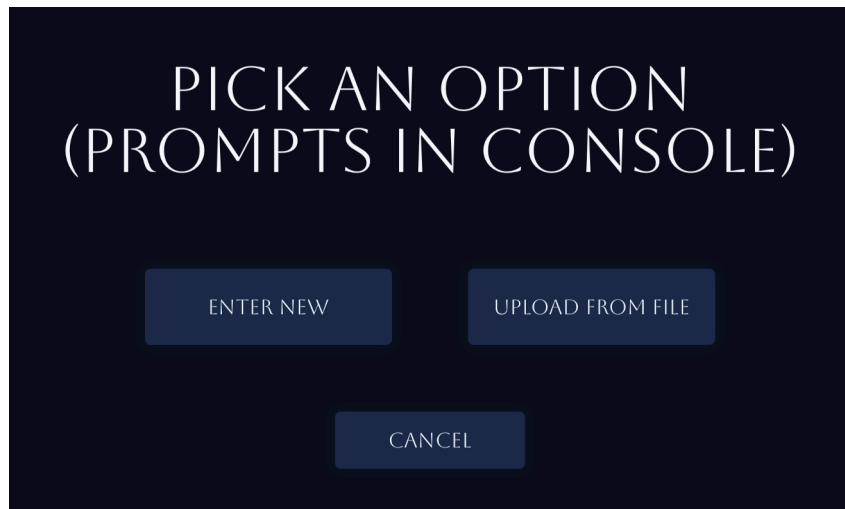


Figure 2: The user is asked for whether they’d like to set up a new configuration, or load one from a JSON file.

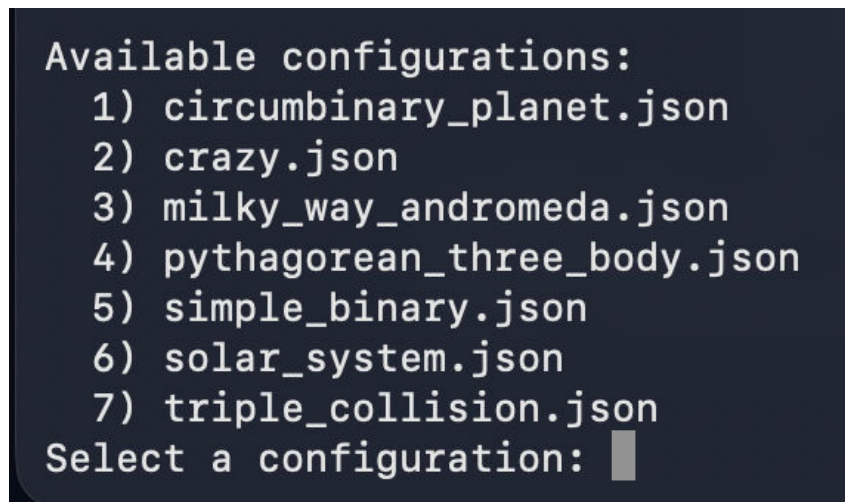


Figure 3: The JSON file options presented in the console for selection.

Once the simulation has been initialised, the user is taken back to the simulation menu where there is now much more information being displayed. You might first decide to go to the top right and switch to 3D mode so you can drag around and zoom in on the objects that have been created. Otherwise, you can move over to the big buttons on the left to either “Run” or “Step”. Run just continuously calls step every frame. Figure 4 shows the 2D perspective, and Figure 5 shows the 3D perspective.

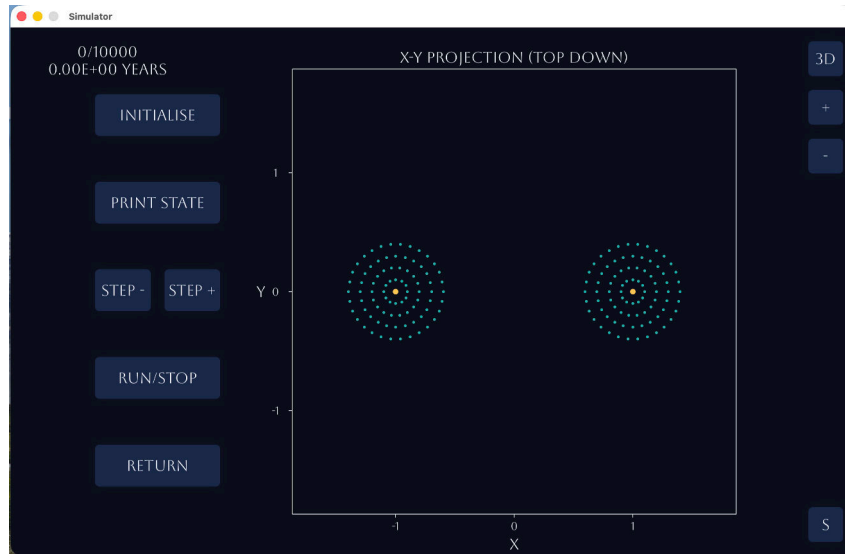


Figure 4: The simulation menu showing a 2D perspective.

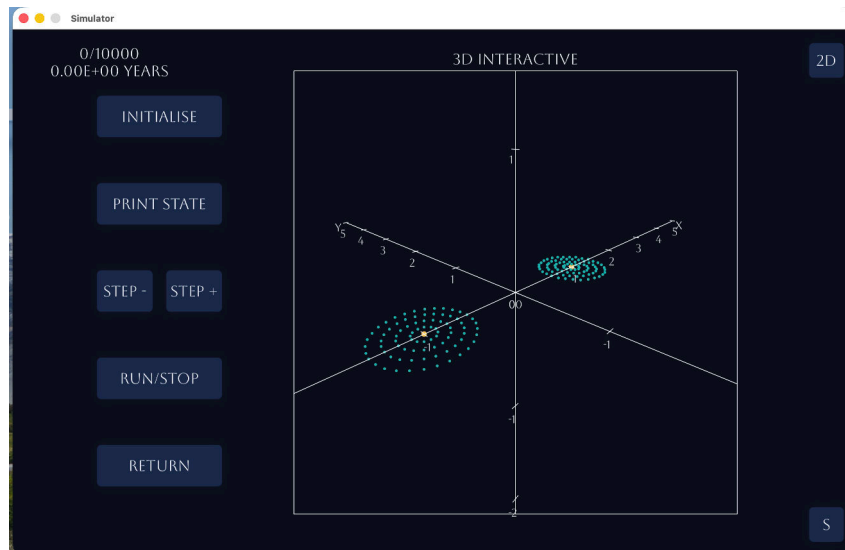


Figure 5: The simulation menu showing a 3D interactive perspective of the same information.

Once the simulation is running, you are free to pause/play it, toggle perspectives, or even print the current state of every object in the system to console. You can also reset with a new configuration. You will also notice that the top left displays the step count of the simulation, as well as the realistic timescale of the simulation. You can use the “S” button in the bottom right to switch between a solar or a galactic timescale. I have written more on this in the Resources section, but the simulation has been normalised such that the gravitational constant is 1. Figure 6 is a frame that was taken while the simulation was running.

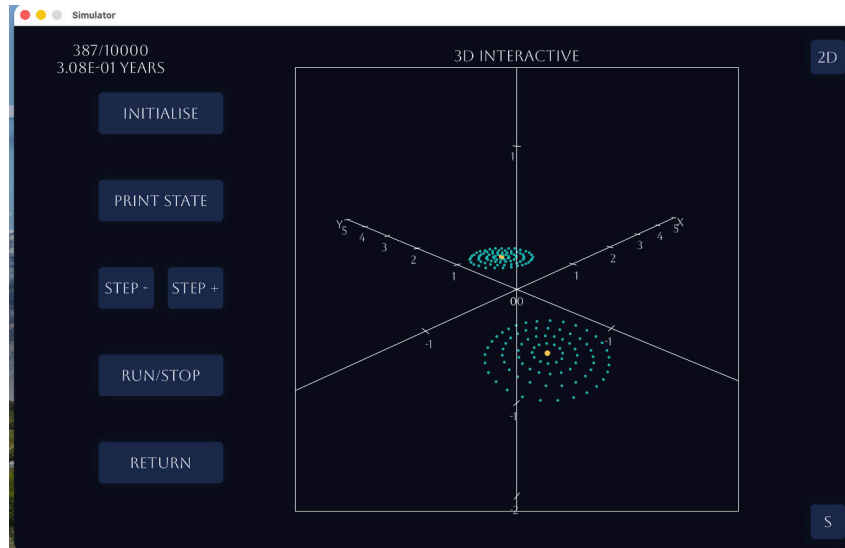


Figure 6: A frame of the simulation menu taken while the simulation is running.

3 Progress

3.1 Successes

A great success of mine in this project, and rather the entire unit, is my learning. I came into FIT1045 knowing nothing about C++. I couldn't even print a string. To be able to redevelop an old Python project of mine into a stable, beautiful replica written in blazing-fast C++ has been incredibly rewarding. I would also like to acknowledge my astrophysics teaching staff who have given me the resources to develop the physics engine (the leapfrog integrator and ring initialisation was reused here).

I was also able to learn how MacOS and Windows handle windows (and using pointers extensively for this!) as well as the matrix transformation that an industry-grade library like OpenGL likely uses. I have a greater appreciation of the software used in everyday cases.

3.2 Challenges

This project was not without its challenges. Every corner in development came with it a set of decisions that needed to be made. I spent what seems like the majority of my development time getting the window management to work on both Mac and Windows. Learning both proprietary libraries was challenging, as they both have very different histories.

Windows historically is littered with endless sets of enums, making memorisation impossible. I had to heavily rely on official documentation to get a working window. It has loads of different data types, sometimes seemingly for the same thing. Windows also gives you less abstraction - getting rounded button corners like with AppKit is much more verbose. I suppose this is a consequence of designing a library to work with decades of legacy hardware.

MacOS, on the other hand, is written for Objective-C. Luckily .mm files allowed me to combine that Objective-C with standard C++, making the use of raw Obj-C very minimal. AppKit is designed only to work on modern Apple computers, meaning it has a generally higher level of abstraction. I found it easier to learn the library than Win32, as it was very internally consistent and gave me easy tools to achieve drop shadows and rounded corners.

Another obvious difference is that MacOS has one of the best text rendering engines in the world. The text on my application looks infinitely better on Mac than it does on Windows.

Other challenges involved the rendering projections. Getting my head around all the different coordinate systems and spaces was initially very challenging. Once I got past the overwhelming feeling of decision paralysis, I just started inputting matrices, multiplying them, and found that the math really just did all the heavy lifting.

Lastly I will acknowledge my use of the console for JSON file selection and configuration initialisation. This is an I/O choice that I made very early on in development, before the window creation, with the original intention to eventually transition this to the GUI. I ended up never getting around to it. I found it to be one of those cases where if it ain't broke, don't fix it. I didn't want to spend hours figuring out how to input text on both Mac and Windows, setting up lots of new menus, all to achieve the same thing, which can be text-based anyway. I acknowledge that a more professional implementation should definitely make use of the GUI for this task.

4 Instructions

The program is designed to compile from the working directory with a single Makefile. The command `make` will compile the binary to `/bin` and immediately run it. The only error you might see is if `/bin` doesn't exist, in which case just create the folder and it will work. It has been tested on Windows 11 and MacOS Tahoe.

5 Video Demonstration

H2 video link (project demonstration): https://drive.google.com/file/d/12n_YYFFNmIBPnJ78bHpd30rWw7QYDe4Z/view?usp=sharing

H+1 video link (defense for high grade): <https://drive.google.com/file/d/1Df4nVtwA0g1tIp3jW9U4zjII58WV14Dd/view?usp=sharing>

Anyone with a Monash email can access these videos. The full Google Drive directory also contains the zipped source code and this PDF, available via <https://drive.google.com/drive/folders/1EMRqKSZ12WwJhELas03MPS8EsJnsQEBM?usp=sharing>

6 Resources

6.1 Unit system

As mentioned prior, the units in the simulation are normalised such that the gravitational constant is 1. In essence, they are meaningless. Two objects spaced one unit apart is not useful unless paired with appropriate scales. We are working with three unique dimensions: time, length, and mass. Given any two scales, we can calculate the third. In this simulation I have chosen to define two different scales: a solar scale and a galactic scale. They are defined by a length and a mass unit, and the time unit is calculated from those. Anyone can go into the source code and add their own unit systems. I used one astronomical unit of length for the solar scale, but someone else might want to use one metre. The simulation only really uses this to display a time to the user, but much can be done with this with further development.

6.2 Font

I included a font file, 'Aboreto.ttf', with the source code, which is set and named in the parameters header file. This can be swapped out for any desired font. I have not tested any other fonts.

7 Conclusion

I am hopeful that my demonstration of coding skills in this project is acceptable to reach the grade of a high distinction. I am personally very pleased with the outcome of my work and I have undergone a very long learning journey. I was successfully able to use a minimal amount of external libraries, and ensured that I was able to learn as much as possible about how such external libraries work under the hood. I am looking forward to questions on my work.